ISO/IEC JTC1 SC22 WG21 P1319R0
Date: 2018-10-08
To: the public
Thomas Köppe <tkoeppe@google.com>

Changes between C++11 and C++14

Abstract

This document enumerates all the major changes that have been applied to the C++ working draft since the publication of C++11, up to the publication of C++14. Major changes are those that were added in the form of a dedicated paper, although not all papers are mentioned individually. The remaining papers are listed summarily below. Issue resolutions from the CWG or LWG issues lists are generally not included, but the containing papers are part of the summary listing. Defect reports that are not part of the published International Standard are marked with a “DR” superscript.

Contents

Removed or deprecated features

Document	Summary	Examples, notes
N3733	Remove `gets()`	This unsafe I/O function from the C library is no longer available.
N3924	Discourage `rand()`	This low-quality random number facility from the C library is discouraged in favour of the `<random>` library.

New features

Document	Summary	Examples, notes
N3649	Polymorphic lambdas	Lambda expressions may now define function templates: `[](auto x) { return x * 2; }`
N3648	Generalized lambda capture	Lambda captures may now be initialized: `[a = 10, b = std::move(b)](){}`
N3651	Variable templates	`template <typename T> constexpr T pi = 4;`
N3472	Binary integer literals	`int pi = 0b101001100;`
N3781	Digit separators	`int pi = 1'010'011'000;`
N3760	`[[deprecated]]` attribute	An attribute to mark deprecated names and entities.
N3656	Creation function `make_unique`	`std::unique_ptr<Base> p = std::make_unique<Derived>(arg1, arg2, arg3);`
N3668	Algorithm `exchange`	`T exchange(T& x, U&& val) { T tmp = std::move(x); x = std::forward<U> (val); return tmp; }`
N3657	Heterogeneous lookup for associative containers	The concept of a “transparent comparator” is introduced, which, if used, adds function *template* overloads `find`/`erase` etc. to associative containers. Convenience predicates `less<>` etc. are provided.
N3659, N3891, LWG 2288	A shared mutex	Reader/writer locking provided by class `shared_timed_mutex` and guard class `shared_lock`.
N3658	Compile-time integer	`integer_sequence`, `index_sequence`, `make_index_sequence` etc., to allow tag

Document	Summary	Examples, notes
	sequences	dispatch on integer packs.
N3654	Quoted strings	`std::cout << std::quoted(s);`
N3642, N3779	User-defined literals	for strings and chrono for complex

Modifications and extensions of existing features

Document	Summary	Examples, notes
N3638	Return type deduction for normal functions	`auto f() { return 10; }` declares `int f();`. In C++11, return type deduction was only available for lambda expressions.
CWG975	Return type deduction for lambdas	The return type of a lambda expression can be deduced even if there are multiple `return` statements. This makes lambda return type deduction work the same way as ordinary function return type deduction (see above). This is intended as a defect report against C++11.
N3669	Constexpr member functions without const	A `constexpr` member function is no longer implicitly `const`. Only for *data* members does `constexpr` imply `const` now.

Document	Summary	Examples, notes
N3652	Relaxing constraints on constexpr functions	`constexpr` functions are far less restricted.
N3653	Member initializers and aggregates	A class with default member initializers may now be an aggregate: `struct X { int a; int b = 10; }; X val = { 5 };`
CWG974	Default arguments for lambdas	Lambda expressions can now have default arguments: `auto f = [] (int a = 10){}; f();` This is intended as a defect report against C++11.
N3778	Sized deallocation	Deallocation functions may now be `void operator delete(void* ptr, size_t len)`
N3664	Mergeable allocation	Adjacent dynamic allocations may now be fused into fewer, larger calls to `operator new`/`operator delete`
N3624	CWG1512: Pointer comparison vs qualification conversions	
N3667	CWG1402: Deleted defaulted ctor	
CWG1288	Direct-list-initialization of references	References can now be initalized correctly using direct-list-initialization: `int & r { n };` Previously, list-initialization would have called for a temporary.

Document	Summary	Examples, notes
		This is intended as a defect report against C++11.
N3922[DR]	New rules for `auto` deduction from braced lists	Previously, `auto a{1, 2, 3}, b{1};` was allowed and both variables were of type `initializer_list<int>`. Now `auto a{1, 2, 3};` is ill-formed and `auto b{1};` declares an `int`. Note that `auto a = {1, 2, 3}, b = {1};` remains unchanged and deduces `initializer_list<int>`.
N3670	Addressing tuples by type	`std::get<int>(my_tuple) = 10;` Ill-formed if the type is not unique.
N3545	`integral_constant::operator()`	Given `integral_constant<int, 123> x;`, `x()` recovers the value `123`.
N3302, N3470, N3469, N3471, N3789	`constexpr` changes	for `complex` for containers for chrono for utilities for functional
N3671	4-iterator sequence operations	Algorithms that operate on two ranges get new overloads that take being and end iterators for *both* ranges rather than requiring the second range to be sufficiently long.

Document	Summary	Examples, notes
N3655	Alias templates for traits	`template <typename T> using foo_t = foo<T>` for traits, e.g. `decay_t`. Part 4 of the paper is not applied.
N3776	`future` destructor	A clarification that the destructor of `future` does not normally block, except when it does.
N3910	CWG1441: signal handlers	A clarification on atomic operations and synchronization in signal handlers.

Miscellaneous

Document	Summary	Examples, notes
N3786	Prohibiting “out of thin air” results	
N3927	Definition of lock-free	
N3323	Contextual conversions	A new notion of “contextual convertibility”

Unlisted papers

The following minor papers are not listed above: N3346, N3644. The following papers contain accepted CWG issues: N3330, N3382, N3383, N3539, N3713,

N3833, N3914. The following papers contain accepted LWG issues: N3318, N3438, N3673, N3522, N3754, N3788, N3930.